Advanced Terraform on Azure
Lab 3 (Guided) - Dynamic Blocks & Deterministic Subnets

Contents

Expected Duration

This lab should take 45-55 minutes to complete

Teaching Points

- Build on your previous configuration by adding a new VNet with subnets using deliberately repetitive baseline code.
- Prepare subnet data in locals for safe, deterministic for_each usage.
- Refactor hard-coded subnet creation into a single map-driven subnet resource.
- Associate NSGs with subnets and prove the design is stable under change.

Before You Begin

You will start this lab using the exemplar code introduced in the last lab, which already includes a resource group, a dynamically generated network security group (NSG) with rules, and a clean naming and hygiene layer in locals.tf.

In this lab, new resources have been added; a Virtual Network (VNet) with subnets. These new components have been introduced in a more manual and inconsistent way. They work, but they break the established patterns of the codebase. Your task is to correct this by refactoring the provided code into a deterministic, for_each-based design that behaves well as the configuration evolves.

Lab Rules

- Work in the lab3/guided folder unless explicitly instructed otherwise.
- Make controlled changes per phase as directed.
- Use the lab3/original folder as a reference to the original code for roll-back if necessary.
- Aim for zero unintended churn – changes should be deliberate and explainable.

Phase 1 – Baseline: VNet with Hard-Coded Subnets

1. Move into the lab3/guided folder.
2. Review the provided files. The named tfvars files are for the challenge phase of this lab. Terraform.tfvars will be used till then.
3. Update **providers.tf** with your subscription id and run **az login**.
4. Open **main.tf** in your editor.
5. Locate the existing **azurerm_resource**_group and **azurerm_network_security_group** resources from Module 2.
6. Scroll down until you find the new vnnet and subnet resources...

```
72  resource "azurerm_virtual_network" "vnet" {
73    name                = "${local.prefix}-vnet"
74    location            = azurerm_resource_group.rg.location
75    resource_group_name = azurerm_resource_group.rg.name
76    address_space       = [var.vnet_address_space]
77    tags                = local.base_tags
78  }
79
80  resource "azurerm_subnet" "app" {
81    name                 = "${local.prefix}-subnet-app"
82    resource_group_name  = azurerm_resource_group.rg.name
83    virtual_network_name = azurerm_virtual_network.vnet.name
84    address_prefixes     = [var.subnet_cidrs["app"]]
85  }
86
87  resource "azurerm_subnet" "data" {
88    name                 = "${local.prefix}-subnet-data"
89    resource_group_name  = azurerm_resource_group.rg.name
90    virtual_network_name = azurerm_virtual_network.vnet.name
91    address_prefixes     = [var.subnet_cidrs["data"]]
92  }
```

7. Open **variables.tf** and verify that the two variables needed to support these new resources exist …

```
variable "vnet_address_space" {
  type        = string
  description = "CIDR for the Virtual Network address space."
}

variable "subnet_cidrs" {
  type        = map(string)
  description = "Map of logical subnet name to CIDR."
}
```

8. Open **terraform.tfvars** and verify that initial values to populate these variables exist …

```
vnet_address_space = "10.0.0.0/16"

subnet_cidrs = {
  app = "10.0.1.0/24"
  data = "10.0.2.0/24"
}
```

9. Consider the pros and cons of this method of provisioning vnets and subnets.

For this first phase, do not change any code. Your task is simply to confirm that the baseline is a working proposition.

10. Run the following commands:

    terraform init
    terraform plan

    Note: If this is the first lab of the day then the initial plan/apply operations may ‘hang’. If nothing happens after 1 minute or so, then perform this one-off fix and retry the plan/apply operation …

    Press Ctrl+c twice to ‘break out’

    Run the following command from you current working directory…

    powershell -ExecutionPolicy Bypass -File C:\qatfadvaz\artifacts\fix\initfix.ps1

The plan should propose the creation of eight resources:

- The resource group and NSG with three rules carried forward from Module 2.
- A virtual network.
- Two subnets: one for app, one for data.

There is nothing *wrong* with this configuration, but the code for the creation of the two subnet resources is clearly repetitive. This is not the best practice.

This phase has set up a working but repetitive baseline: the NSG is dynamic and well-structured, but the subnets are still defined as two separate, hand-written resources.

Phase 2 - New subnet local variable

Preliminaries

11. Open **locals.tf** in lab3/guided.
12. Review the existing locals used for naming, tagging, and NSG rule cleaning.
13. Add a new local variable to normalize the subnet_cidrs variable values ...

```
subnet_cidrs_clean = {
 for name, cidr in var.subnet_cidrs :
 lower(trimspace(name)) => trimspace(cidr)
}
```

This creates a cleaned view of your RAW subnet map variable, where the keys are lower-cased and trimmed, and the CIDR strings are trimmed of any accidental whitespace.

14. Run **terraform plan** again

There should be no changes. You have only introduced a new local variable, not altered any resources. This is a safe, zero-impact data-shaping step.

You have prepared a deterministic map (subnet_cidrs_clean) that will be safe to use as a for_each source. This mirrors the approach used for NSG rules in Module 2.

Phase 3 – Refactor Subnets to a for_each Resource

15. Return to **main.tf** and find the two hard-coded subnet resources.
16. Replace the two separate subnet resources with a single for_each-driven resource...

```
resource "azurerm_subnet" "subnet" {
 for_each          = local.subnet_cidrs_clean
 name              = "${local.prefix}-subnet-${each.key}"
 resource_group_name  = azurerm_resource_group.rg.name
 virtual_network_name = azurerm_virtual_network.vnet.name
 address_prefixes     = [each.value]
}
```

This resource will create one subnet per entry in local.subnet_cidrs_clean, naming each subnet consistently based on the shared prefix and the cleaned key.

17. Verify this refactor ...

```
terraform plan
terraform apply --auto-approve
```

18. Use the portal to verify the resource creation.

    You have replaced repetitive, hand-written subnet resource provisioning with a map-driven, deterministic for_each subnet resource. This matches the dynamic pattern used for NSG rules earlier in the course.

Phase 4 – Associate NSG with Subnets

We must now associate the NSG resource with the newly provisioned subnets.

19. In **main.tf**, add a new resource to associate the NSG with every subnet created by azurerm_subnet.subnet:

```
resource "azurerm_subnet_network_security_group_association" "subnet" {
 for_each                  = azurerm_subnet.subnet
 subnet_id                 = each.value.id
 network_security_group_id = azurerm_network_security_group.nsg.id
}
```

    This creates one NSG association per subnet, keyed by the same cleaned names (app, data, and any others you may add).

20. Verify ..

```
terraform plan
terraform apply --auto-approve
```

    This should now deploy one association per subnet;

    - azurerm_subnet_network_security_group_association.subnet["app"]
    - azurerm_subnet_network_security_group_association.subnet["data"]

    NSG associations are now driven by the same map that drives the subnets themselves. Adding or removing a subnet automatically adds or removes the corresponding association.

Phase 5 – Reorder and Extend Subnets via tfvars

21. Open **terraform.tfvars** and locate the **subnet_cidrs** map values.

Reordering and sloppy input

22. First, experiment with reordering the existing entries and introduce “sloppy” input (leading and trailing spaces).

23. In **terraform.tfvars**, change this:

```
subnet_cidrs = {
 app = "10.0.1.0/24"
 data = "10.0.2.0/24"
}
```

to this:

```
subnet_cidrs = {
 data = "  10.0.2.0/24"
 app = "10.0.1.0/24  "
}
```

24. Run **terraform plan**

    Because the subnets are driven by local.subnet_cidrs_clean (which trims spaces and lower-cases keys), the plan should be a no-op. Reordering a map should not cause churn when using for_each with deterministic keys.

Extending with a new subnet

25. Next, add a new subnet entry …

```
subnet_cidrs = {
 data  = "  10.0.2.0/24"
 app  = "10.0.1.0/24  "
 endpoints = " 10.0.3.0/24"
}
```

26. Run **terraform plan**

    You should see a **new subnet** proposed and a matching NSG association

    The existing app and data subnets should remain unchanged.

    This confirms that:

- Reordering the map does **not** cause churn.
- Extra whitespace is safely handled by local.subnet_cidrs_clean.
- Adding a new entry adds exactly one new subnet and association.

Introducing a subtle but fatal error

We will now deliberately introduce a **broken CIDR** to see a limitation of our current hygiene-only approach.

27. Change the app subnet to use an invalid prefix length:

```
subnet_cidrs = {
 data = "  10.0.2.0/24"
 app = "10.0.1.0/24  "
 endpoints = " 10.0.3.0/34"
}
```

28. Run **terraform plan**

You should see Terraform accepts the input and will therefore attempt to create the subnets during an apply operation, because as far as Terraform is concerned this value is "just a string." The real problem appears when the Azure provider (or API) validates the CIDR.

29. Try **terraform apply --auto-approve**

    Azure should return an error complaining about an **invalid address prefix** for the app subnet. The exact wording may vary, but the key point is that:

    - Our trimming and normalisation logic (subnet_cidrs_clean) happily passes the invalid string through.
    - The failure only occurs **late** when the provider/API rejects 10.0.3.0/34.

30. Observe the error returned by Azure for the app subnet. Note that:

    - The failure is not caught by Terraforms own type system (it is still "a string").
    - The error message is **low-level** and appears only at apply time, not at variable validation time.

    You have proven that:

    - Hygiene (trimming and normalising) protects you from **churn and cosmetic issues**.
    - It does **not** protect you from **semantically invalid values** such as a malformed CIDR.

    This sets up an important guardrail requirement: we need **validation** on subnet_cidrs so that invalid CIDRs are caught **early and clearly**, instead of leaking through to Azure and failing at apply time. The next phase will address this gap.

Phase 6 – Add CIDR Guardrail for Subnets

31. Ensure your **terraform.tfvars** still contains the broken subnet definition.
32. Open **variables.tf** in lab3/guided and locate the subnet_cidrs variable.
33. Add a validation block to ensure every subnet CIDR is syntactically valid. Extend subnet_cidrs variable definition ...

```
...
 validation {
  condition = alltrue([
   for cidr in values(var.subnet_cidrs) :
   can(cidrhost(trimspace(cidr), 0))
  ])
  error_message = "All subnet_cidrs values must be valid."
 }
...
```

Here, cidrhost() is used as a **probe**: if Terraform cannot calculate a host address, the CIDR is invalid, and the can() wrapper will return false.

34. With the broken app CIDR still set to /34, run **terraform plan** You should now see an immediate terraform validation error
35. In **terraform.tfvars**, fix the app subnet to a valid CIDR and re-run the plan:

```
subnet_cidrs = {
 data  = "  10.0.2.0/24"
 app  = "10.0.1.0/24  "
 endpoints = " 10.0.3.0/24"
}
```

36. Run **terraform plan** There should be no issues
37. Run **terraform apply --auto-approve** to deploy the new subnet and association
38. Use the portal to verify the resource creation

    You have now:

- Combined **hygiene** (locals that trim and normalise) with **guardrails** (variable validation).
- Ensured that "bad but syntactically string-like" values such as 10.0.2.0/34 are rejected early, with a human-readable message.
- Mirrored the same approach used earlier in the NSG rules: cleanse the data, then enforce rules that keep bad values out of the cloud.

This is the core pattern of the course; **Shape inputs + validate them = deterministic, safe terraform.**

Phase 7 - Challenge: Validating VNet and Subnet Addressing

At the end of this lab, the Terraform configuration allows a wide range of network inputs that are syntactically valid but can still fail during deployment when Azure applies its own platform rules. In this challenge, your goal is to strengthen input validation so that common network design mistakes are detected early, during terraform plan, rather than later during deployment.

39. As a starting point, review **variables.tf**. You will notice a commented-out variable named **approved_vnet_cidrs**. Azure does not accept all valid CIDR ranges for Virtual Networks, and allowing users to supply arbitrary address spaces can lead to unexpected deployment failures.
40. As part of this challenge, **uncomment and use this variable** to restrict the VNet address space to one of the known, approved values.

41. Your task is then to update **variables.tf** so that:
    A. vnet_address_space must be a valid CIDR **and** must be one of the approved values defined in **approved_vnet_cidrs**
    B. all subnets are at least **/29** in size (Azure minimum)
    C. all subnets are smaller than the VNet (subnet prefix length must be greater than the VNet prefix length)
42. When complete, invalid configurations should fail fast during terraform plan, with clear and meaningful error messages. Use the supplied .tfvars files to test both valid and invalid scenarios, and observe how early validation improves feedback compared to relying solely on Azure's deployment-time checks.
43. Review the named tfvars files ...

    good.tfvars - Complies with 41 A,B and C
    bad_unapproved_vnet.tfvars - Non-compliant with 41 A
    bad_subnet_too_small.tfvars - Non-compliant with 41 B
    bad_subnet_too_small.tfvars - Non-compliant with 41 C

44. Run the following tests to confirm that your changes to variables.tf result in one success and four errors...

    terraform plan --var-file=good.tfvars
    terraform plan --var-file=bad_unapproved_vnet.tfvars
    terraform plan --var-file=bad_subnet_too_small.tfvars
    terraform plan --var-file=bad_subnet_same_size_as_vnet.tfvars

Scope note: In this challenge, focus on validating **CIDR structure and mask sizes** (for example, ensuring masks are valid, subnet sizes are appropriate, and subnets are smaller than the VNet). You are not expected to calculate or validate the exact IP address ranges themselves. If you find yourself attempting detailed IP arithmetic, you are going beyond the scope of this exercise.

Proposed solution **variables.tf** file is at **lab3\solutions\guided**.

Bonus.

Exemplar code is provided at **lab3\solutions\guided\exemplar** that does fully validate **exact IP** address ranges and subnet mask values. It also demonstrates use of the **null provider**. This is not an official part of the training because it highlights **excessive code complexity**. In real-world environments, teams are rarely allowed to choose IP address ranges freely. Instead, address space allocation is usually managed centrally through **IP Address Management (IPAM)** or a dedicated **core networking team**. These systems maintain a single source of truth for which address ranges are available, allocated, or reserved, preventing overlaps and enforcing organisational standards. Terraform is then

used to deploy infrastructure using **pre-approved allocations**, rather than to calculate or validate address space on the fly.

A common Terraform pattern that reflects this approach is a central network module that owns Virtual Network creation, subnet layout, and address space decisions. Application teams consume the outputs of this module (such as subnet IDs or VNet IDs) but never supply CIDRs themselves. In the next few modules, you will begin moving toward this model by separating networking concerns into a dedicated Network module, while the NSG logic remains in a separate Security module. This shift reduces the need for complex validation logic and reinforces a key Terraform principle: well-designed modules prevent invalid configurations by construction, rather than by exhaustive validation.

45. If you wish to run this exemplar ..

    If applicable; Destroy any resources created during this lab.

    Navigate to lab3\solutions\exemplar

    Read the README.md

    Update **providers.tf** with your **subscription id**

    terraform init/plan

    Adjust the vnet_address_space and subnet_cidrs values in terraform.tfvars to test

Phase 8 – Clean up

46. During this lab you deployed resources into Azure. Destroy these ahead of the next module using **terraform destroy --auto-approve**

Copyright